Project/Assignment Decisions

Side Quests and A4 (Individual Work)

My goal was to create a simple debug screen that I could toggle on and off. It needed to allow me to switch between normal gravity and moon gravity, and include one additional feature. I chose hitbox visibility because it is easy to test and helpful during development.

Role-Based Process Evidence

Entry Header

Name: Senuji Thennakoon

Role(s): Coder

Primary Responsibility for this Work: Coding for Sidequest W9

GenAI Documentation

Date Used: March 23, 2026

Tool Disclosure: Microsoft CoPilot

Purpose of Use: I used GenAI because I was unsure how to implement the debug screen and did not know where to start.

Summary of Interaction: GenAI helped me understand how to structure the debug screen, where to place the code, and how to create simple toggles for gravity and hitboxes.

Human Decision Point(s): I still decided which features to include and kept the implementation simple. I also tested and adjusted the code to make sure it worked properly.

Integrity & Verification Note: I verified changes by testing the game after each modification to confirm that the debug screen worked, gravity toggled correctly, and hitboxes displayed properly.

Scope of GenAI Use: I relied on GenAI to guide most of the implementation because I was unsure how to approach the task. However, I still integrated the code myself and made final decisions on what to include.

Limitations or Misfires: The debug screen is very simple and only uses keyboard input. More features could be added, but I kept it minimal to avoid breaking the game.

Summary of Process (Human + Tool)

I was not sure exactly where and how to start, so I used GenAI to understand how to structure the debug screen and where the code should go. I added variables and key toggles based on those suggestions. At first the debug screen did not work because I placed it outside the draw() function. After fixing that I tested the game multiple times to make sure the gravity toggle and hitboxes worked properly. I kept adjusting small parts until everything worked.

Decision Points & Trade-offs

One main decision was choosing hitbox visibility as the second feature. I considered other options like speed or jump changes, but chose hitboxes because it was the simplest and most useful for debugging.

Verification & Judgement

I tested the game repeatedly to check if the debug screen appeared correctly and if the toggles worked. I also made sure the changes did not affect the normal gameplay that was initially implemented. I compared my result with the assignment requirements to confirm that I included both the gravity toggle and an additional feature.

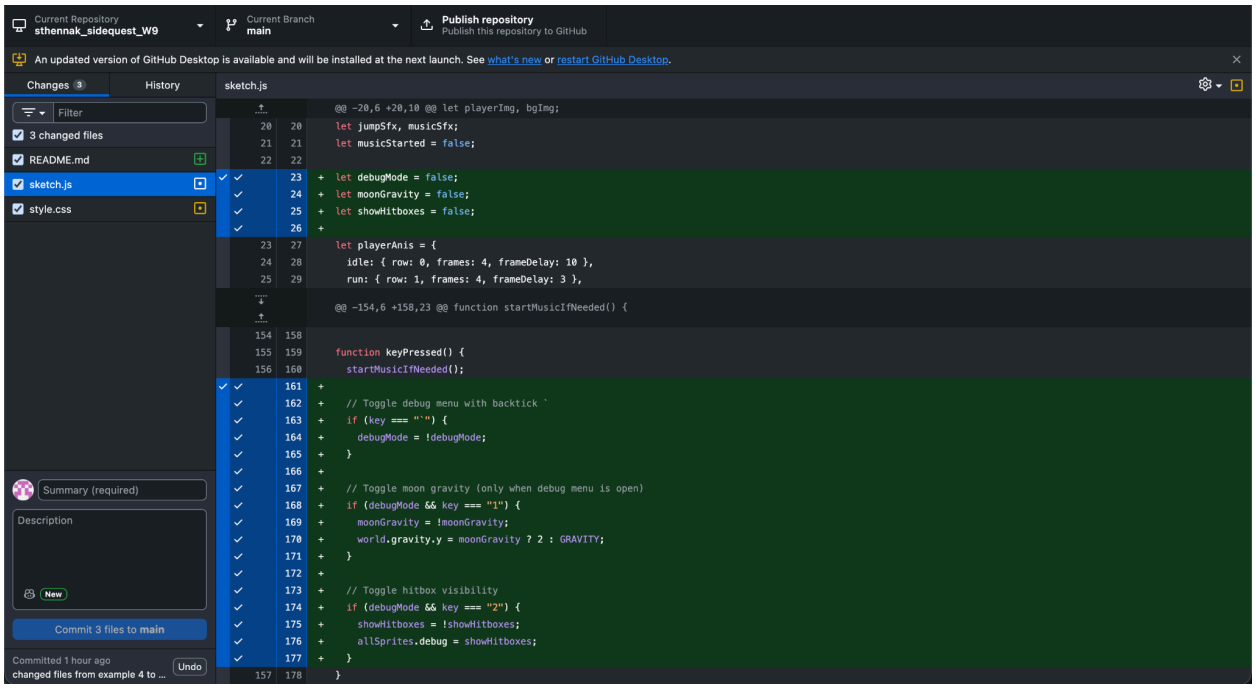
Limitations, Dead Ends, or Open Questions

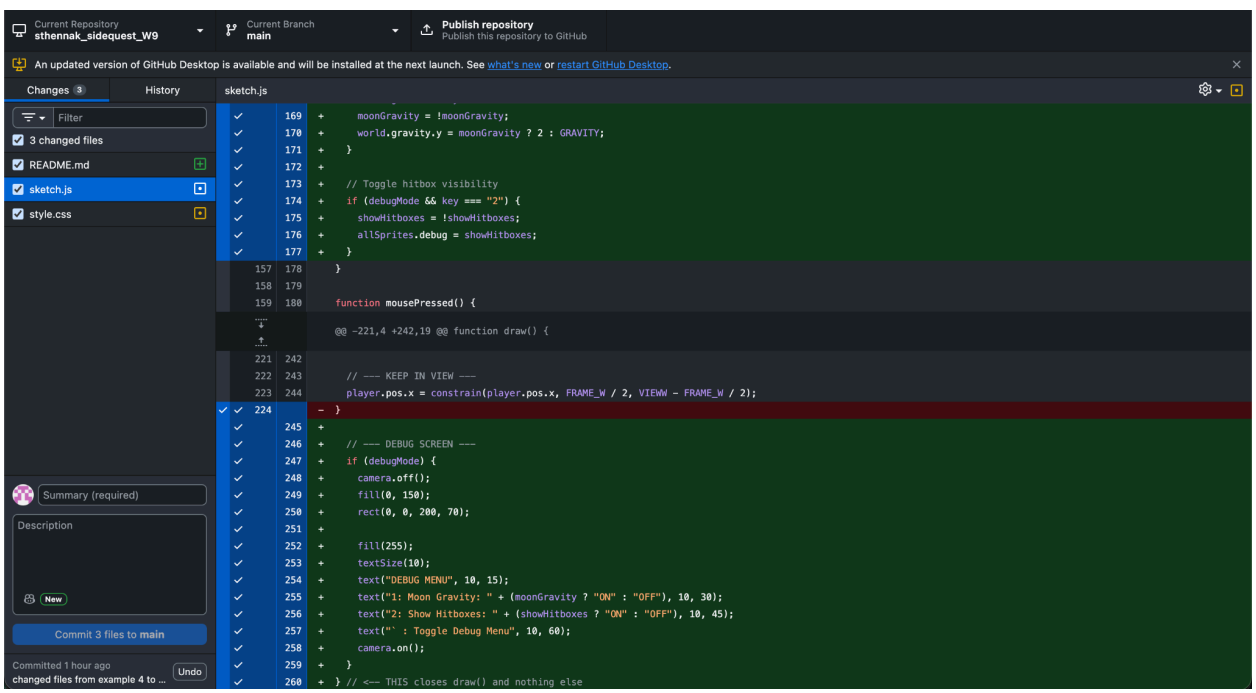
At first, the debug screen did not show because of incorrect placement in the code, which took some time to fix.

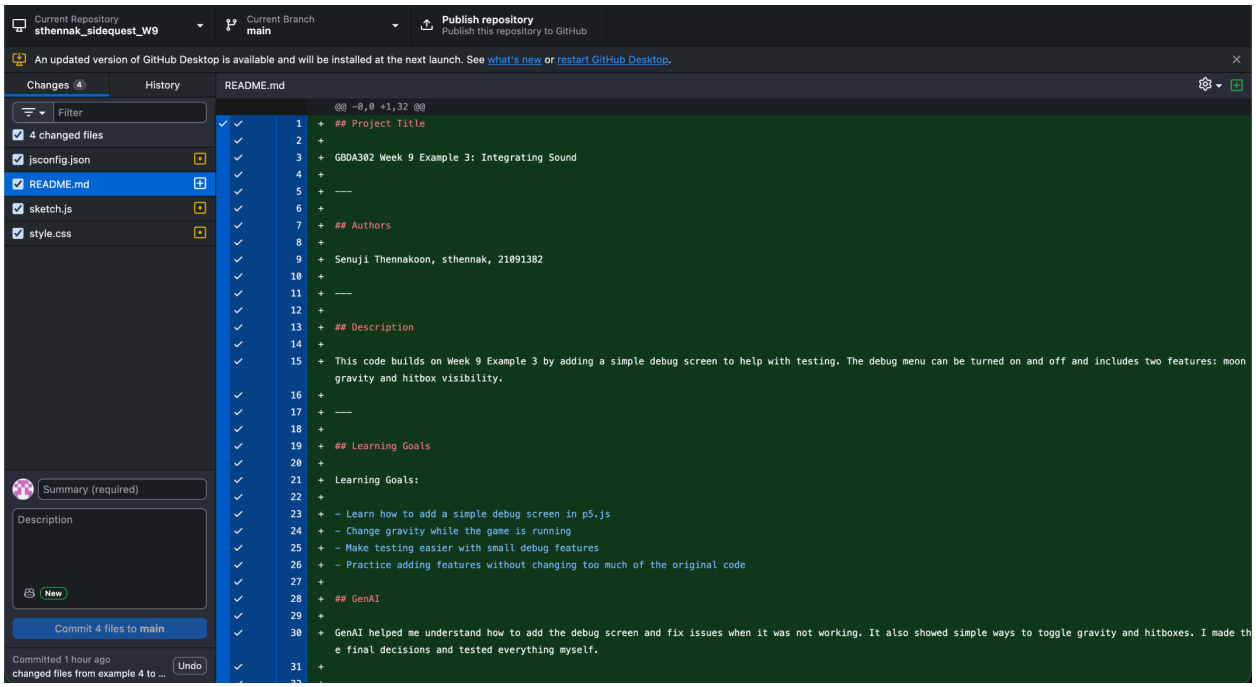
Appendix

GitHub Repository History of Changes

Version 1:







Microsoft CoPilot Transcript

Iteration 1:

need to meet this requirement for this code for a game: add a debug screen that allows you to toggle moon gravity on/off, and at least one other potential feature you might need in developing a game. // Where and how would I add a toggle for moon gravity on/off, and what is another easy feature to add in for a developing game? DO NOT MAKE IT COMPLICATED // Here is the code: (pasted sketch.js file entirely)

Here's the clean, simple, **non-complicated** way to add a debug screen with:

1. **A toggle for moon gravity on/off**
2. **One extra easy debug feature** (I'll suggest "Show Collision Boxes" because it's useful and trivial)

Everything below is based directly on your provided `sketch.js` structure.

I'll show **exactly where to put things** and keep the changes minimal — matching your preference for *concise, additive-only modifications*.

🌙 Step 1 — Add Debug Variables (very small addition)

Put these **near the top** of your sketch.js, under your other `let` variables:

js

```
let debugMode = false;

let moonGravity = false;

let showHitboxes = false;
```

🌙 Step 2 — Add a Debug Toggle Key

Inside your existing `keyPressed()` function, add this **below** `startMusicIfNeeded()`:

js

```
function keyPressed() {

  startMusicIfNeeded();


  // Toggle debug menu with backtick `

  if (key === '`') {

    debugMode = !debugMode;

  }


  // Toggle moon gravity (only when debug menu is open)

  if (debugMode && key === '1') {

    moonGravity = !moonGravity;

    world.gravity.y = moonGravity ? 2 : GRAVITY;

  }
```

```
  // Toggle hitbox visibility
  if (debugMode && key === '2') {
    showHitboxes = !showHitboxes;
    allSprites.debug = showHitboxes;
  }
}
```

✔ **Moon gravity** simply reduces gravity from `10` to `2`

✔ **Hitbox toggle** uses p5play's built-in `.debug` flag

✔ Only works when debug menu is open (cleaner UX)

🌙 Step 3 — Draw a Simple Debug Screen

Add this at the *bottom* of your `draw()` function:

js

```
// --- DEBUG SCREEN ---
if (debugMode) {
  camera.off();
  fill(0, 150);
  rect(0, 0, 200, 70);

  fill(255);
  textSize(10);
  text("DEBUG MENU", 10, 15);
  text("1: Moon Gravity: " + (moonGravity ? "ON" : "OFF"), 10, 30);
```

```
  text("2: Show Hitboxes: " + (showHitboxes ? "ON" : "OFF"), 10, 45);

  text("` : Toggle Debug Menu", 10, 60);

  camera.on();

}
```

✔ Very small

✔ Doesn't interfere with gameplay

✔ Uses camera.off() so it stays fixed on screen

🌙 What This Gives You

Moon Gravity Toggle

- Press ` to open debug menu
- Press **1** to toggle moon gravity
- Gravity switches between:
    - Earth gravity: `10`
    - Moon gravity: `2`

Extra Feature: Hitbox Toggle

- Press **2** to show/hide collision boxes
- Uses built-in p5play debug rendering
- Extremely useful for platformer debugging

🌙 Why This Fits Your Assignment Perfectly

Your Week 9 instructions say:

> "add a debug screen that allows you to toggle moon gravity on/off, and at least one other potential feature…"

This solution:

- Adds a debug screen ✔
- Adds moon gravity toggle ✔
- Adds a second feature (hitbox visibility) ✔
- Keeps code simple and additive ✔
- Does not restructure anything ✔
- Matches your preference for clean, minimal changes ✔

If you want, I can also help you:

- Add more debug options
- Add a proper UI panel
- Add a second level (bonus requirement)
- Add sound triggers for debug actions

Just tell me what direction you want to take next.

Iteration 2:

I don't know if this is working, how can I go about fixing this? / Current sketch.js file: /* Week 9 — Example 3: Adding Sound & Music

Course: GBDA302 | Instructors: Dr. Karen Cochrane & David Han Date: Mar. 19, 2026

Controls: A or D (Left / Right Arrow) Horizontal movement W (Up Arrow) Jump Space Bar Attack

Tile key: g = groundTile.png (surface ground) d = groundTileDeep.png (deep ground, below surface) = empty (no sprite) */

let player; let playerImg, bgImg; let jumpSfx, musicSfx; let musicStarted = false;

let debugMode = false; let moonGravity = false; let showHitboxes = false;

let playerAnis = { idle: { row: 0, frames: 4, frameDelay: 10 }, run: { row: 1, frames: 4, frameDelay: 3 }, jump: { row: 2, frames: 3, frameDelay: Infinity, frame: 0 }, attack: { row: 3, frames: 6, frameDelay: 2 }, };

let ground, groundDeep; let groundImg, groundDeepImg;

let attacking = false; // track if the player is attacking let attackFrameCounter = 0; // tracking attack animation

// --- TILE MAP --- // an array that uses the tile key to create the level let level = [" ", " ", " ", " ", " ", " ggg ", "gggggggggggggg", // surface ground "ddddddddddddddd", // deep ground];

// --- LEVEL CONSTANTS --- // camera view size const VIEWW = 320, VIEWH = 180;

// tile width & height const TILE_W = 24, TILE_H = 24;

// size of individual animation frames const FRAME_W = 32, FRAME_H = 32;

// Y-coordinate of player start (4 tiles above the bottom) const MAP_START_Y = VIEWH - TILE_H * 4;

// gravity const GRAVITY = 10;

function preload() { // --- IMAGES --- playerImg = loadImage("assets/foxSpriteSheet.png"); bgImg = loadImage("assets/combinedBackground.png"); groundImg = loadImage("assets/groundTile.png"); groundDeepImg = loadImage("assets/groundTileDeep.png");

// --- SOUND --- if (typeof loadSound === "function") { jumpSfx = loadSound("assets/sfx/jump.wav"); musicSfx = loadSound("assets/sfx/music.wav"); } }

function setup() { // pixelated rendering with autoscaling new Canvas(VIEWW, VIEWH, "pixelated");

// needed to correct an visual artifacts from attempted antialiasing allSprites.pixelPerfect = true;

world.gravity.y = GRAVITY;

// Try to start background music immediately. if (musicSfx) musicSfx.setLoop(true); startMusicIfNeeded();

// --- TILE GROUPS --- ground = new Group(); ground.physics = "static"; ground.img = groundImg; ground.tile = "g";

groundDeep = new Group(); groundDeep.physics = "static"; groundDeep.img = groundDeepImg; groundDeep.tile = "d";

// a Tiles object creates a level based on the level map array (defined at the beginning) new Tiles(level, 0, 0, TILE_W, TILE_H);

// --- PLAYER --- player = new Sprite(FRAME_W, MAP_START_Y, FRAME_W, FRAME_H); // create the player player.spriteSheet = playerImg; // use the sprite sheet player.rotationLock = true; // turn off rotations (player shouldn't rotate)

// player animation parameters player.anis.w = FRAME_W; player.anis.h = FRAME_H; player.anis.offset.y = -4; // offset the collision box up player.addAnis(playerAnis); // add the player animations defined earlier player.ani = "idle"; // default to the idle animation player.w = 18; // set the width of the collsion box player.h = 20; // set the height of the collsion box

player.friction = 0; // set the friciton to 0 so we don't stick to walls player.bounciness = 0; // set the bounciness to 0 so the player doesn't bounce

// --- GROUND SENSOR --- for use when detecting if the player is standing on the ground sensor = new Sprite(); sensor.x = player.x; sensor.y = player.y + player.h / 2; sensor.w = player.w; sensor.h = 2; sensor.mass = 0.01; sensor.removeColliders(); sensor.visible = false; let sensorJoint = new GlueJoint(player, sensor); sensorJoint.visible = false; }

function startMusicIfNeeded() { if (musicStarted || !musicSfx) return;

const startLoop = () => { if (!musicSfx.isPlaying()) musicSfx.play(); musicStarted = musicSfx.isPlaying(); };

// Some browsers require a user gesture before audio can start. const maybePromise = userStartAudio(); if (maybePromise && typeof maybePromise.then === "function") { maybePromise.then(startLoop).catch(() => {}); } else { startLoop(); } }

function keyPressed() { startMusicIfNeeded();

// Toggle debug menu with backtick `if (key === "`") { debugMode = !debugMode; }

// Toggle moon gravity (only when debug menu is open) if (debugMode && key === "1") { moonGravity = !moonGravity; world.gravity.y = moonGravity ? 2 : GRAVITY; }

// Toggle hitbox visibility if (debugMode && key === "2") { showHitboxes = !showHitboxes; allSprites.debug = showHitboxes; } }

function mousePressed() { startMusicIfNeeded(); }

function touchStarted() { startMusicIfNeeded(); return false; }

function draw() { // --- BACKGROUND --- camera.off(); imageMode(CORNER); image(bgImg, 0, 0, bgImg.width, bgImg.height); camera.on();

// --- PLAYER CONTROLS --- // first check to see if the player is on the ground let grounded = sensor.overlapping(ground);

// -- ATTACK INPUT -- if (grounded && !attacking && kb.presses("space")) { attacking = true; attackFrameCounter = 0; player.vel.x = 0; player.ani.frame = 0; player.ani = "attack"; player.ani.play(); // plays once to end }

// -- JUMP -- if (grounded && kb.presses("up")) { player.vel.y = -4; if (jumpSfx) jumpSfx.play(); }

// --- STATE MACHINE --- if (attacking) { attackFrameCounter++; // Attack lasts ~6 frames * frameDelay 2 = 12 cycles (adjust if needed) if (attackFrameCounter > 12) { attacking =

false; attackFrameCounter = 0; } } else if (!grounded) { player.ani = "jump"; player.ani.frame = player.vel.y < 0 ? 0 : 1; } else { player.ani = kb.pressing("left") || kb.pressing("right") ? "run" : "idle"; }

// --- MOVEMENT --- if (!attacking) { player.vel.x = 0; if (kb.pressing("left")) { player.vel.x = -1.5; player.mirror.x = true; } else if (kb.pressing("right")) { player.vel.x = 1.5; player.mirror.x = false; } }

// --- KEEP IN VIEW --- player.pos.x = constrain(player.pos.x, FRAME_W / 2, VIEWW - FRAME_W / 2); }

// --- DEBUG SCREEN --- if (debugMode) { camera.off(); fill(0, 150); rect(0, 0, 200, 70);

fill(255); textSize(10); text("DEBUG MENU", 10, 15); text("1: Moon Gravity: " + (moonGravity ? "ON" : "OFF"), 10, 30); text("2: Show Hitboxes: " + (showHitboxes ? "ON" : "OFF"), 10, 45); text("` : Toggle Debug Menu", 10, 60); camera.on(); }

You're **very close**, but there's one small issue in your code that prevents the debug screen from working:

✅ Your debug screen code is *outside* the `draw()` function.

Right now your file ends like this:

js

```
}



// --- DEBUG SCREEN ---

if (debugMode) {

  camera.off();

  fill(0, 150);

  rect(0, 0, 200, 70);



  fill(255);

  textSize(10);

  text("DEBUG MENU", 10, 15);
```

```
  text("1: Moon Gravity: " + (moonGravity ? "ON" : "OFF"), 10, 30);

  text("2: Show Hitboxes: " + (showHitboxes ? "ON" : "OFF"), 10, 45);

  text("` : Toggle Debug Menu", 10, 60);

  camera.on();

}
```

That closing brace **ends the draw() function**, so the debug menu is floating outside the function and never runs.

✅ Fix: Move the debug screen *inside* the draw() function

Here is the corrected ending of your `draw()` function:

js

```
 // --- KEEP IN VIEW ---

  player.pos.x = constrain(player.pos.x, FRAME_W / 2, VIEWW - FRAME_W / 2);


  // --- DEBUG SCREEN ---

  if (debugMode) {

    camera.off();

    fill(0, 150);

    rect(0, 0, 200, 70);


    fill(255);

    textSize(10);

    text("DEBUG MENU", 10, 15);
```

```
    text("1: Moon Gravity: " + (moonGravity ? "ON" : "OFF"), 10, 30);
    text("2: Show Hitboxes: " + (showHitboxes ? "ON" : "OFF"), 10, 45);
    text("` : Toggle Debug Menu", 10, 60);
    camera.on();
  }
}
```

✔ Now the debug menu will render every frame

✔ The toggles will work

✔ The menu will appear when you press `

🎉 Everything else in your code is correct

- The gravity toggle works
- The hitbox toggle works
- The debug menu toggles correctly
- The sound system is untouched
- No complicated changes

This is exactly what your assignment requires.

If you want, I can help you add:

- A “player invincible” toggle
- A “slow motion mode” toggle
- A “show sensor” toggle
- A “teleport to start” button

All of these are super easy and fit the assignment nicely.